МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Информационные технология»

Лабораторная работа №2
по дисциплине «Введение в информационные технологии» по теме:
«Введение в язык программирования Python»

Выполнил студент группы:
Номер группы: БВТ2604
ФИО: Пименов Иван Владимирович
Проверил: Павликов А. Е

Москва 2026

Оглавление

Введение.....……...3

Выполнение.....4-14

Выводы 15

Цель и задачи работы:

Цель работы: изучить принципы создания и использования функций в языке Python, научиться передавать данные в функции и возвращать результаты, понять области видимости переменных, освоить работу со списками и кортежами, срезами и встроенными функциями, а также получить практический опыт реализации базовых алгоритмов обработки последовательностей и применения lambda-функций. В ходе выполнения лабораторной работы необходимо научиться разбивать программу на логически завершённые функции, использовать параметры и возвращаемые значения, обрабатывать последовательности данных и самостоятельно реализовывать простейшие алгоритмы поиска, подсчёта и агрегирования.

Выполнение:

Задание 1: Создание и вызов функций

Мы создали три функции: greet(name) для вывода приветствия, square(number) для возврата квадрата числа и max_of_two(a, b) для определения большего из двух значений без использования встроенной функции max(). Мы вызвали каждую функцию минимум дважды с разными аргументами и зафиксировали полученные результаты.

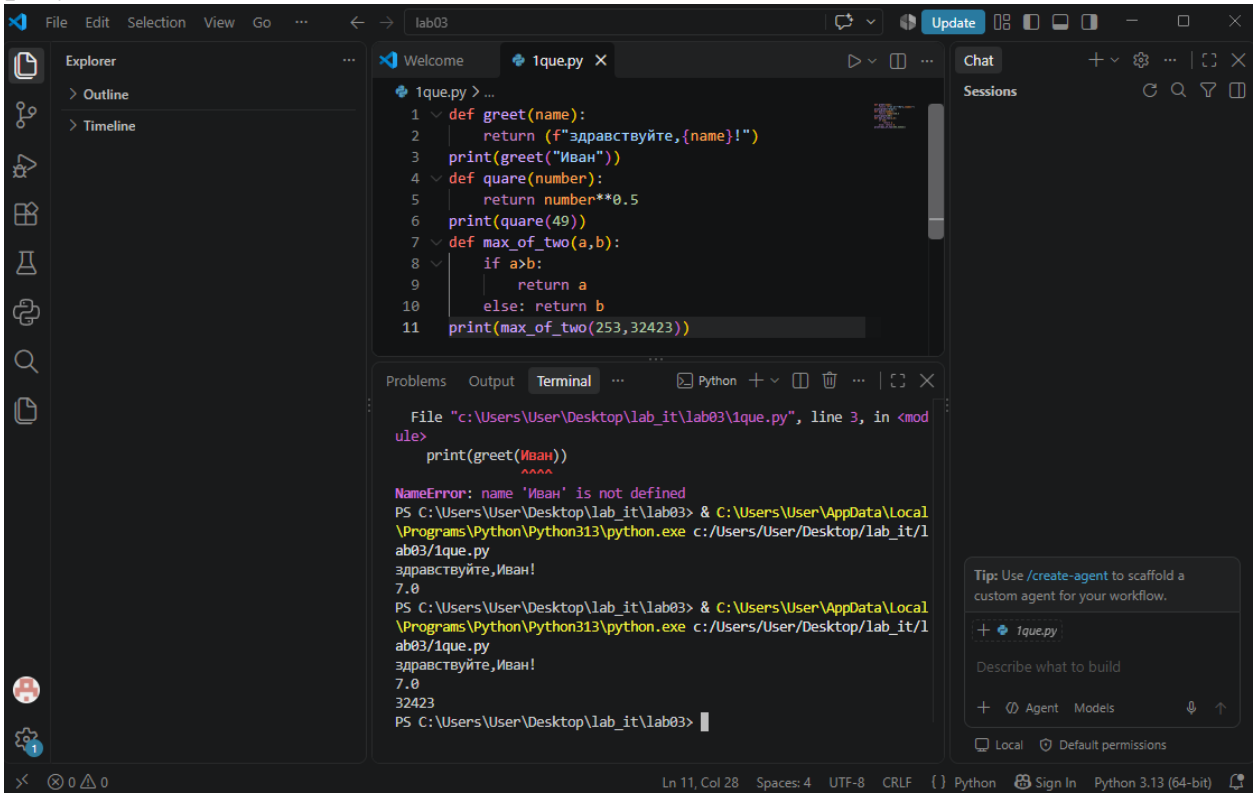


Рисунок 1: Проверка 1

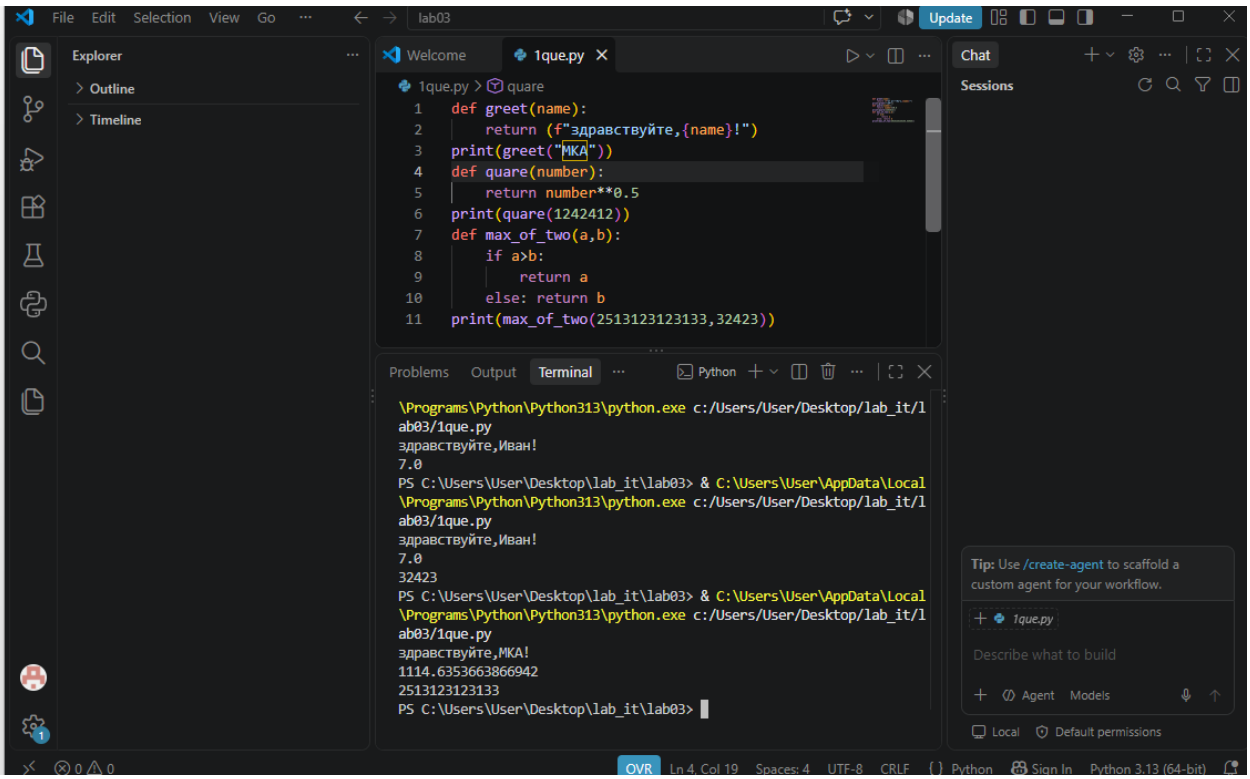


Рисунок 2: Проверка 2

Задание 2: Аргументы и возвращаемые значения

Мы реализовали функции calculate_rectangle(width, height) и calculate_perimeter(width, height) для расчёта площади и периметра прямоугольника. Также мы создали функцию describe_rectangle(width, height, unit="см"), в которой использовали значение по умолчанию для единицы измерения. Мы продемонстрировали вызов describe_rectangle() как с явным указанием аргумента unit, так и без него, и зафиксировали различия в выводе

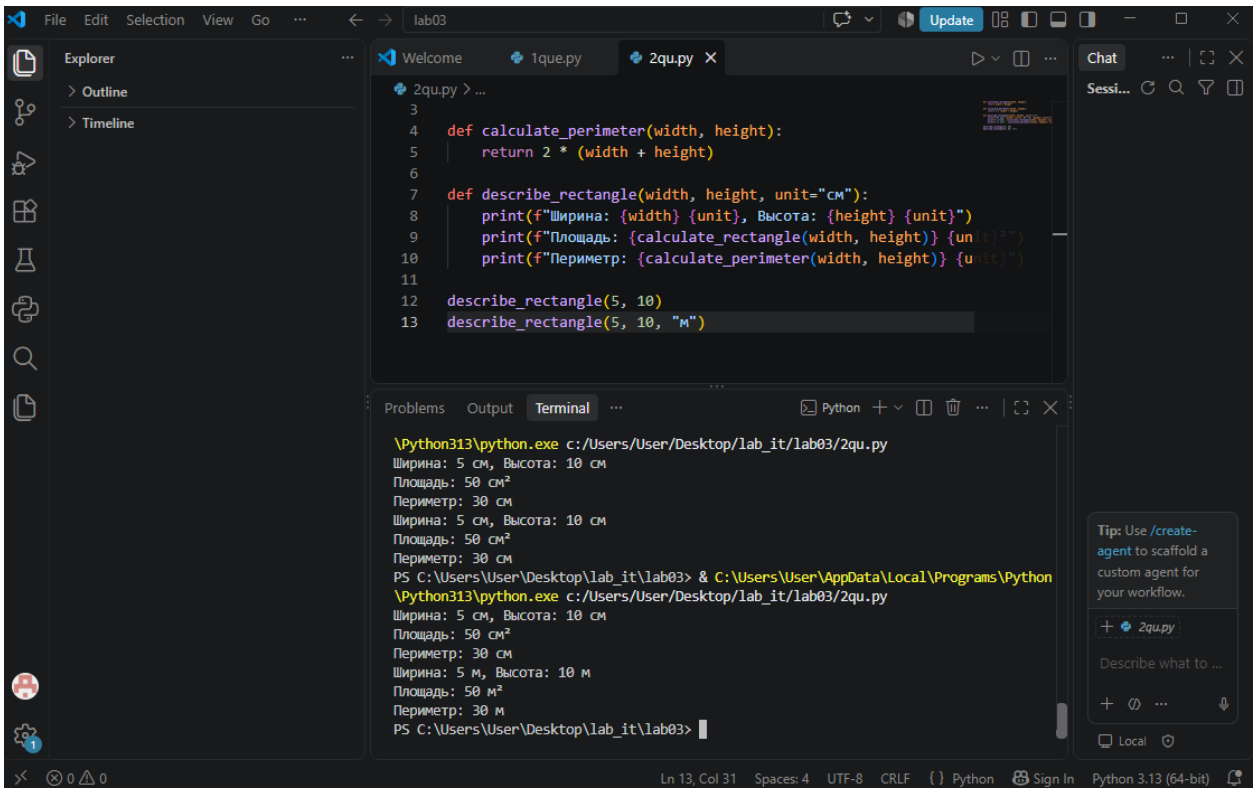


Рисунок 3: Проверка

Задание 3: Области видимости переменных

Мы выполнили предоставленный фрагмент кода с переменной value и функцией example(), зафиксировали результаты вывода и объяснили, что внутри функции создаётся локальная переменная, которая не влияет на глобальную. Мы добавили в функцию ещё одну локальную переменную, попытались обратиться к ней вне функции и зафиксировали ошибку; объяснили, что локальные переменные недоступны за пределами своей области видимости.

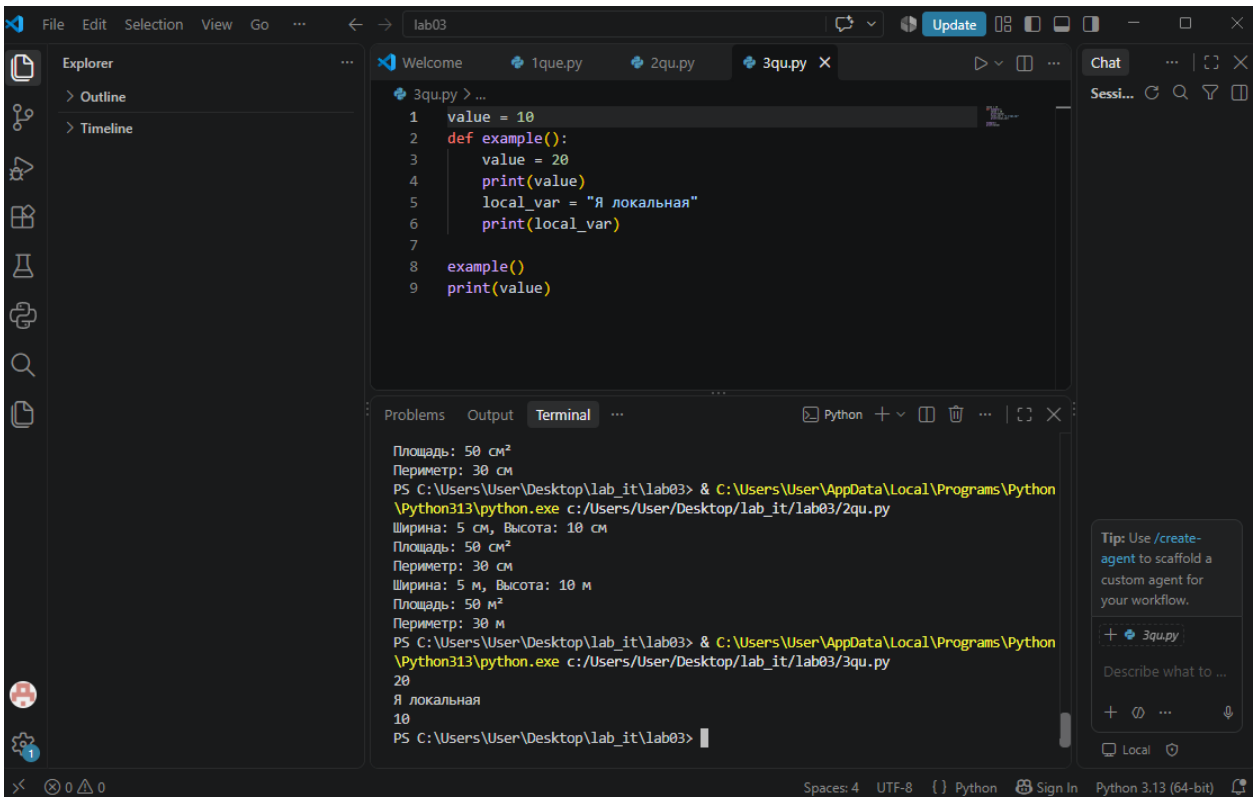


Рисунок 4: Полученный результат

Задание 4: Работа со списками

Мы создали список, содержащий не менее 10 целых чисел. Мы вывели первый, третий и последний элементы списка, изменили значение второго элемента, добавили новый элемент с помощью метода append(), удалили один из элементов через remove() или pop(). В завершение мы вывели итоговый список и количество его элементов.

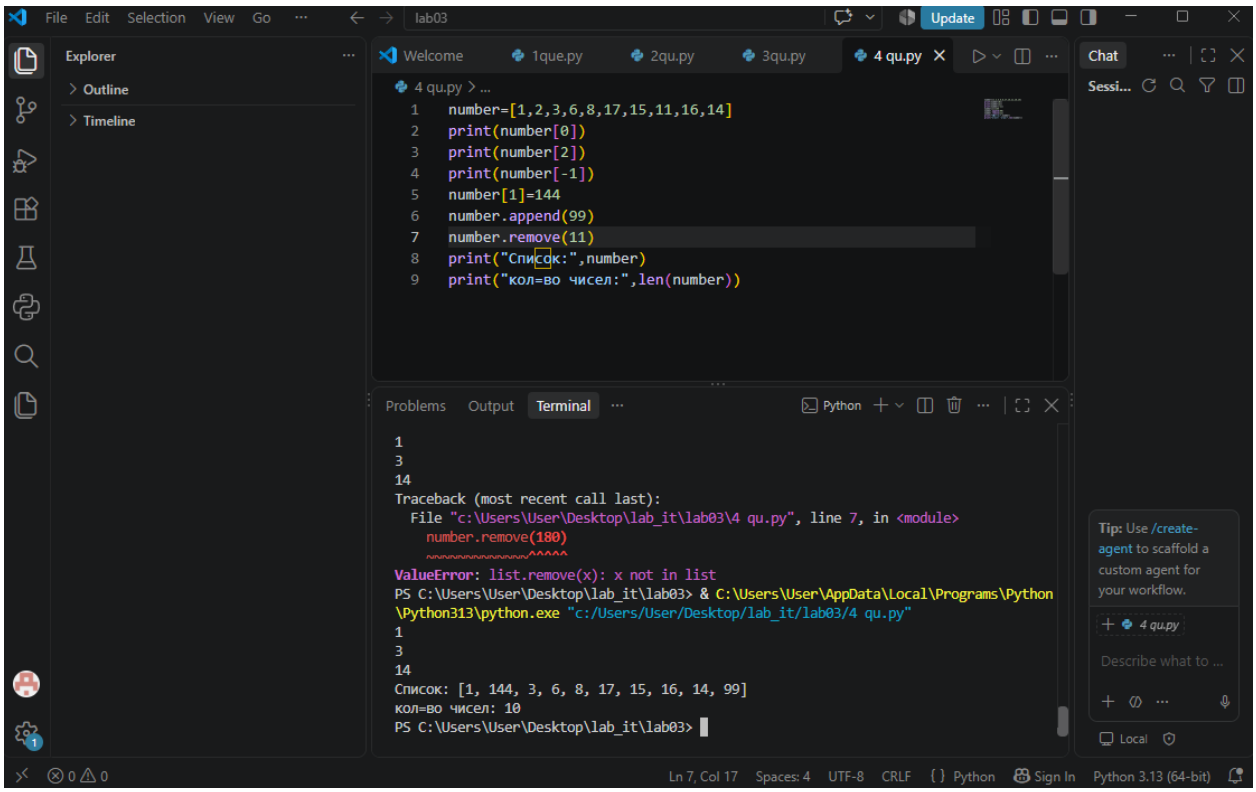


Рисунок 5: Итоговый список

Задание 5: Использование срезов

Мы использовали список numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] и с помощью срезов получили и вывели: первые три элемента, последние три элемента, элементы с 3-го по 7-й, каждый второй элемент, а также список в обратном порядке. Мы зафиксировали все результаты и убедились в корректности работы срезов.

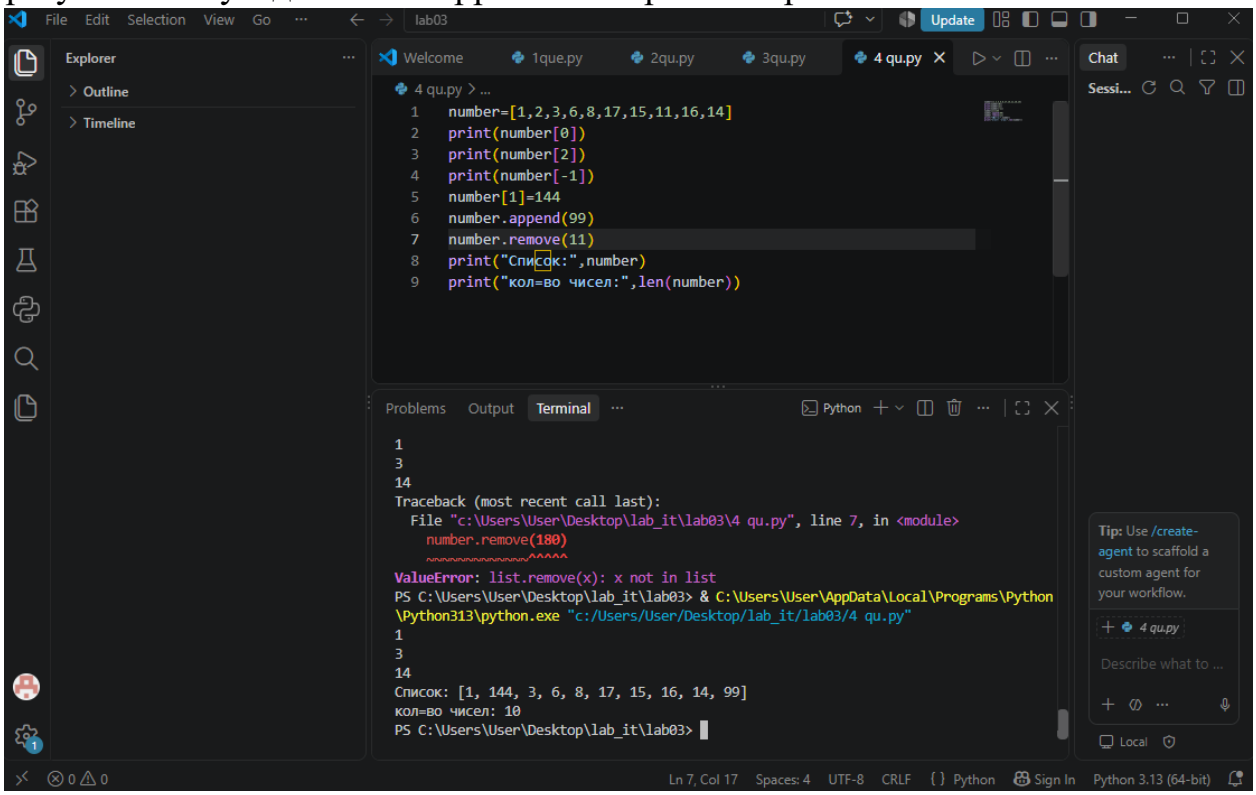


Рисунок 6: Работа со списками

Задание 6: Работа с кортежами

Мы создали кортеж measurements с не менее чем 8 числовыми значениями, вывели отдельные элементы и срезы. Мы попытались изменить элемент кортежа и зафиксировали возникающую ошибку; объяснили неизменяемость кортежей в Python. Мы распаковали первые два значения кортежа в отдельные переменные, преобразовали кортеж в список, изменили элемент и вернули результат обратно в кортеж.

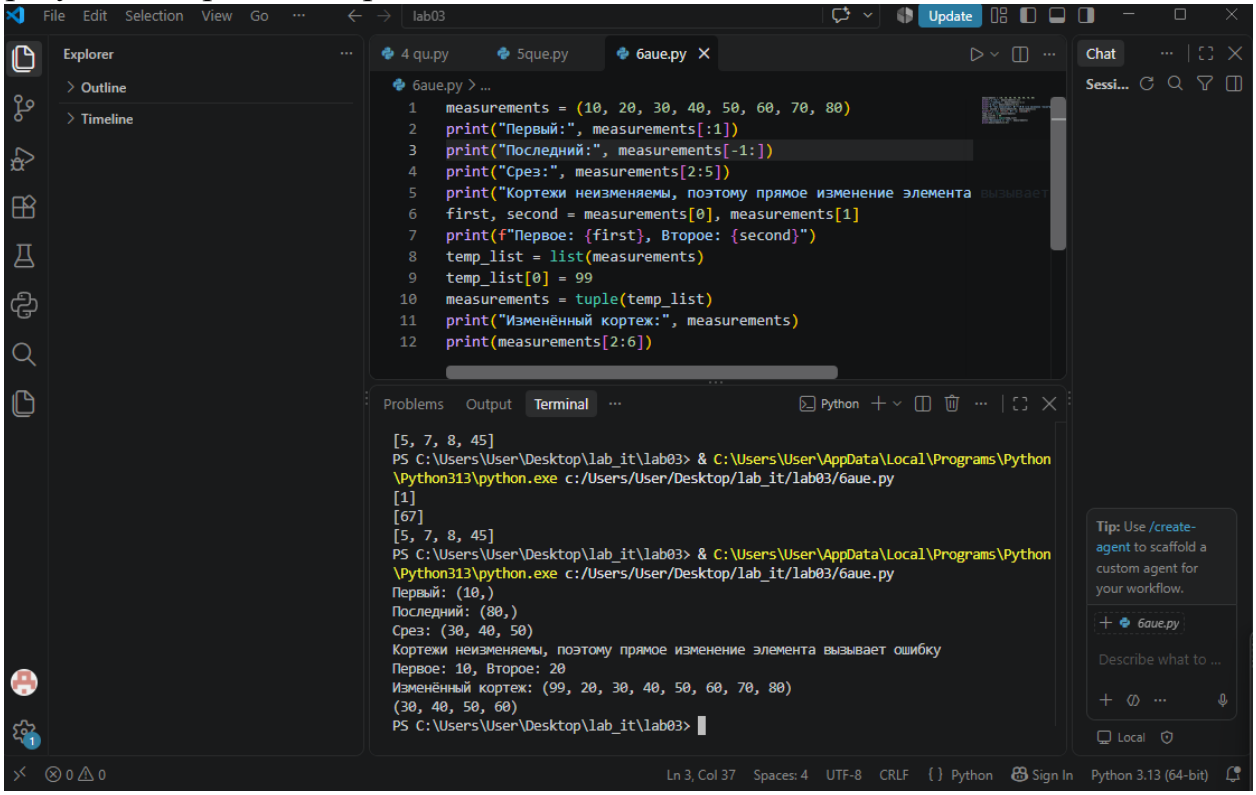


Рисунок 7: Работа с кортежами

Задание 7: Встроенные функции

Для ранее созданного списка чисел мы определили количество элементов, сумму, минимальное и максимальное значения, а также вычислили среднее арифметическое. Мы получили отсортированную копию списка с помощью функции sorted() и отсортировали сам список методом sort(). Мы сравнили результаты и объяснили различие: sorted() возвращает новый список, а sort() изменяет исходный.

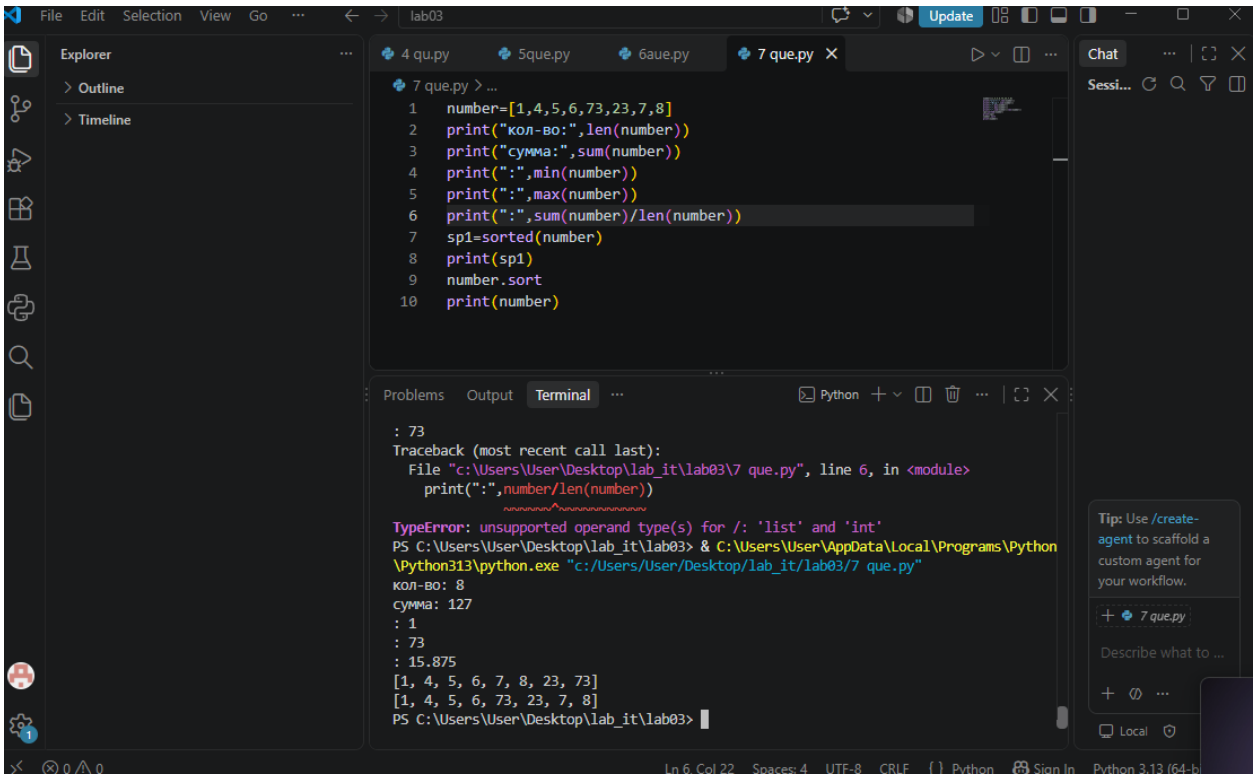


Рисунок 8: Встроенные функции

Задание 8: Реализация собственных алгоритмов

Мы реализовали собственные функции calculate_sum(numbers), find_min(numbers) и find_max(numbers) без использования встроенных функций sum(), min() и max(). Для этого мы применили циклы и условные операторы для последовательного перебора элементов. Мы сравнили полученные значения с результатами встроенных функций и убедились в их совпадении.

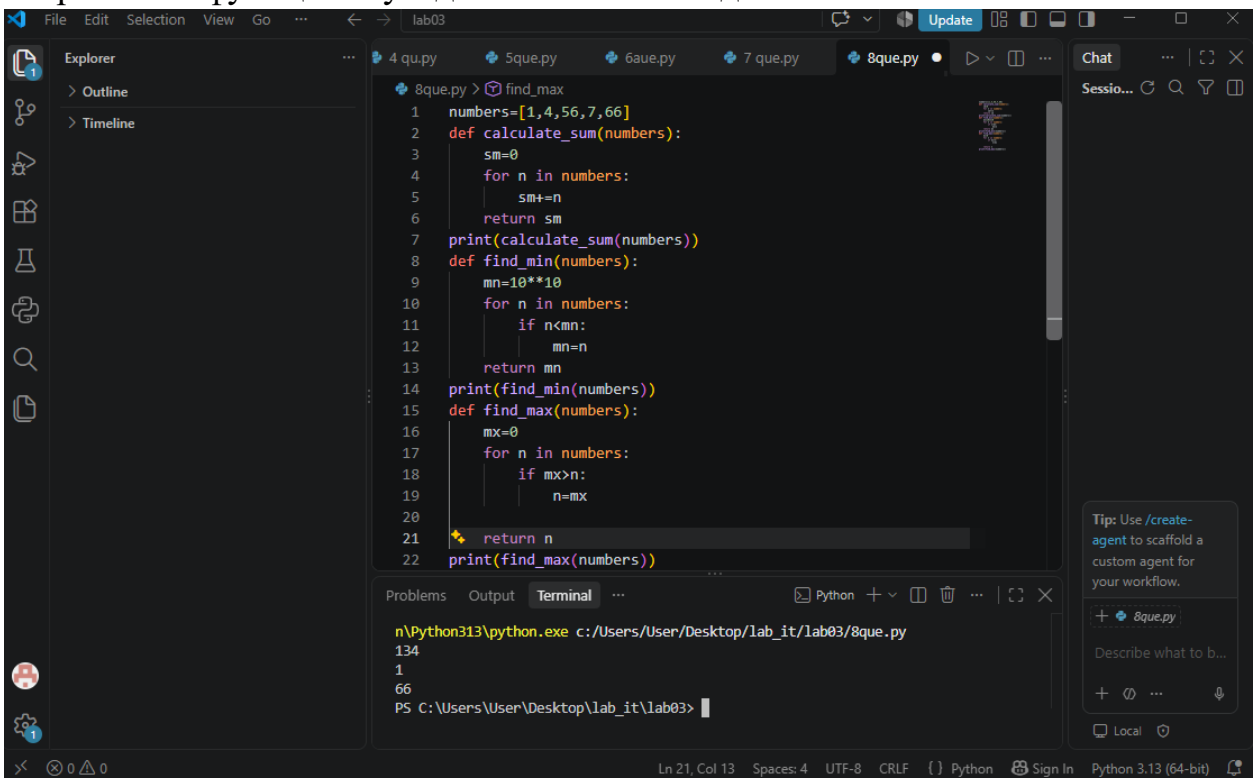


Рисунок 9: Реализация функций без встроенных

Задание 9: Линейный поиск

Мы реализовали функцию find_element(numbers, target), которая последовательно просматривает список и возвращает индекс первого найденного элемента либо -1, если элемент отсутствует. Мы проверили работу алгоритма на примере существующего и отсутствующего значения и зафиксировали корректность работы функции.

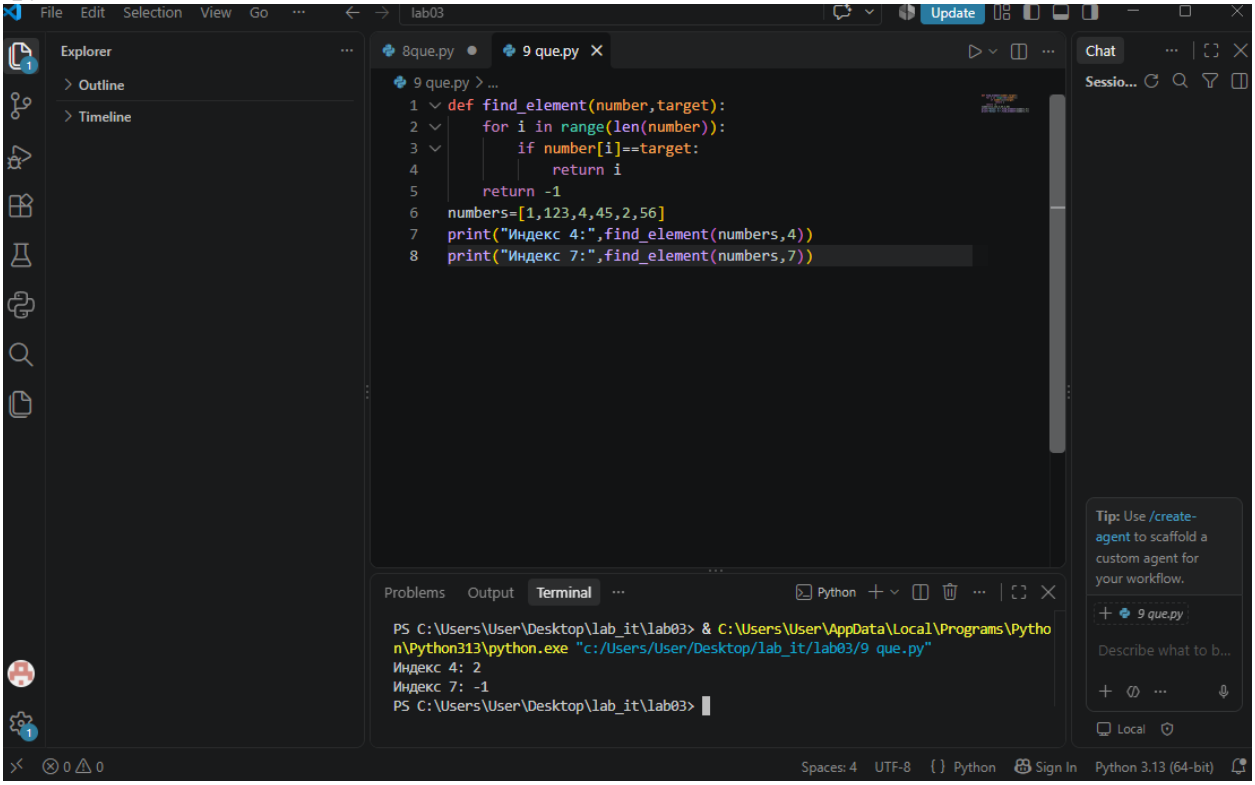


Рисунок 10: Реализация функции

Задание 10: Возврат нескольких значений

Мы создали функцию analyze(numbers), которая принимает список чисел и возвращает кортеж из минимального значения, максимального значения и среднего арифметического. Мы вызвали функцию, распаковали возвращённый кортеж в три отдельные переменные и вывели результаты на экран.

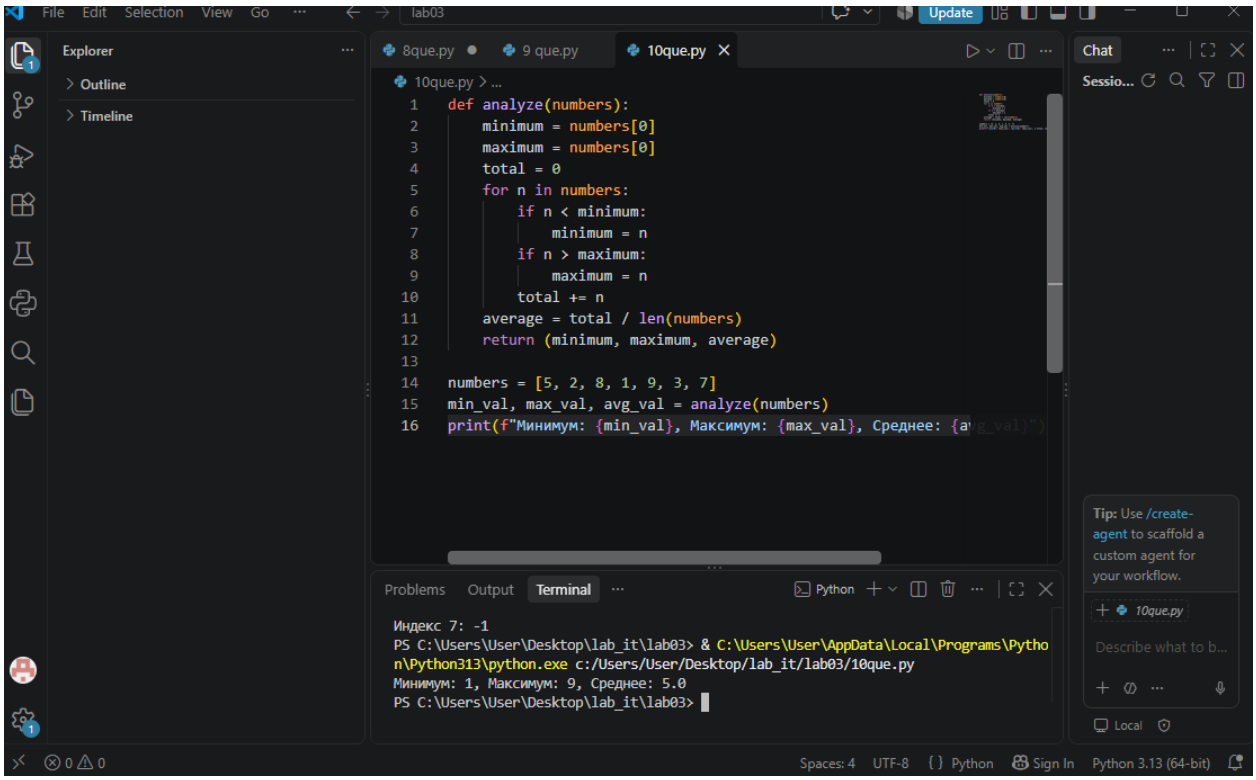


Рисунок 11: Возврат

Задание 11: Lambda-функции

Мы использовали список numbers = [-10, 3, -2, 8, -5] и сначала отсортировали его обычным способом с помощью sorted(). Затем мы отсортировали список по абсолютному значению элементов, применив параметр key и lambda-функцию lambda x: abs(x). Мы сравнили два результата и объяснили назначение lambda-функции: она позволяет задать простое правило сортировки без объявления отдельной функции.

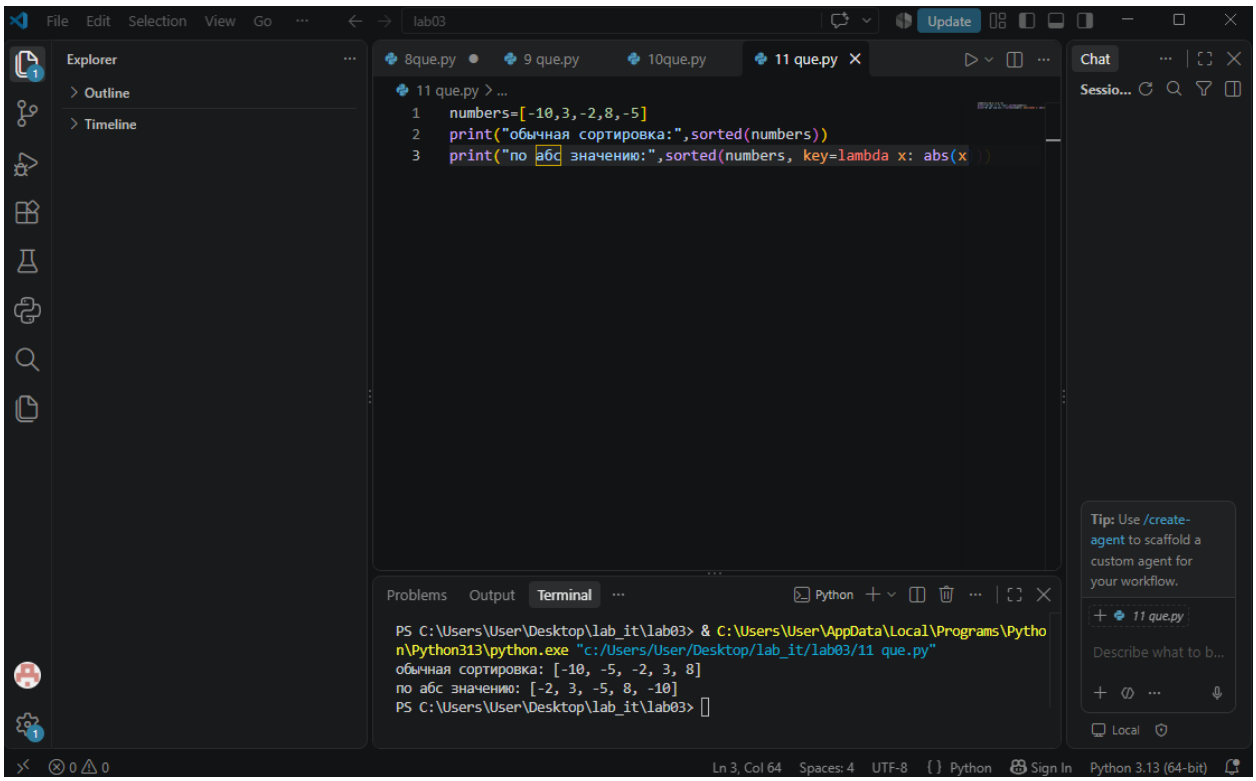


Рисунок 12: Lambda функция

Задание 12: Самостоятельная работа

Мы разработали программу для анализа набора результатов измерений, хранящихся в списке из не менее чем 10 числовых значений. Мы реализовали отдельные функции: calculate_average(), find_minimum(), find_maximum(), count_above_average(), find_value() и analyze_measurements(). В функциях calculate_average(), find_minimum() и find_maximum() мы не использовали встроенные функции sum(), min() и max(), реализовав алгоритмы вручную. Функция analyze_measurements() возвращает кортеж с минимумом, максимумом и средним значением; мы распаковали его в отдельные переменные. Программа выводит количество измерений, минимальное и максимальное значение, среднее значение, количество значений выше среднего и результат поиска заданного пользователем значения. Мы протестировали программу и зафиксировали корректную работу всех функций.

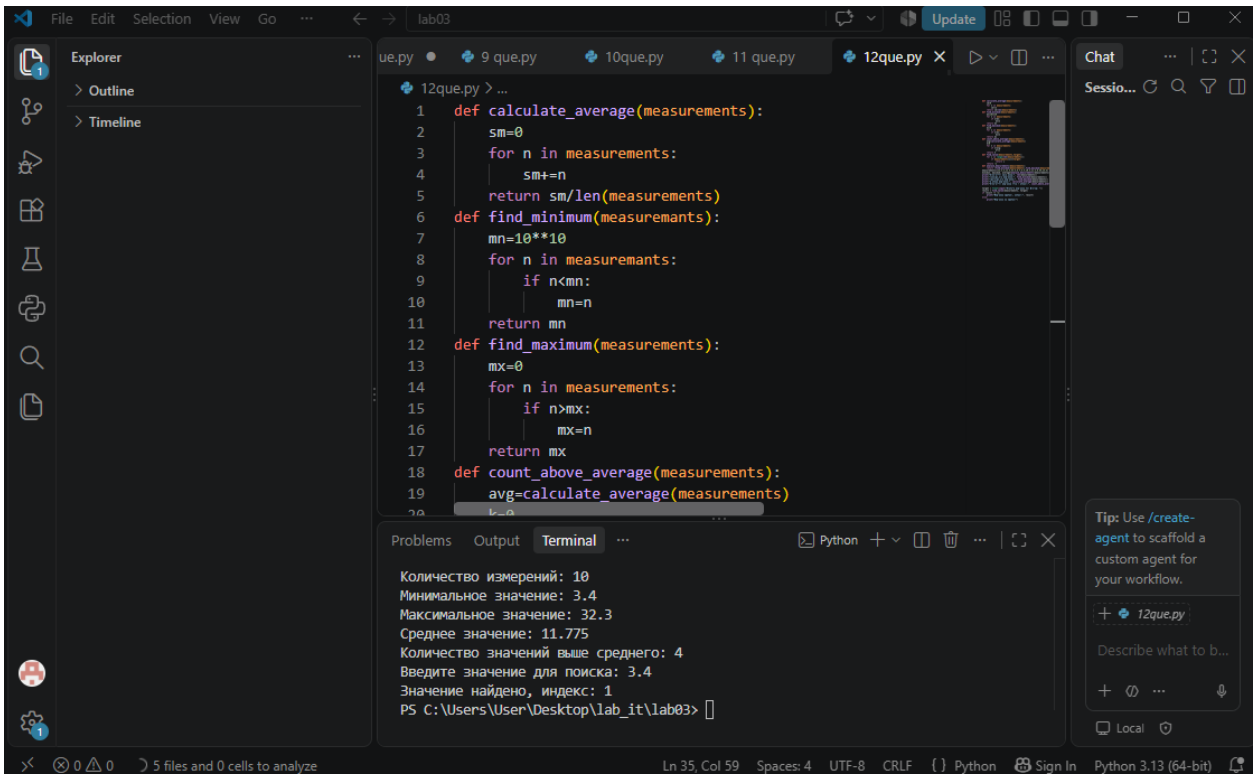


Рисунок 13: Программа 1 часть

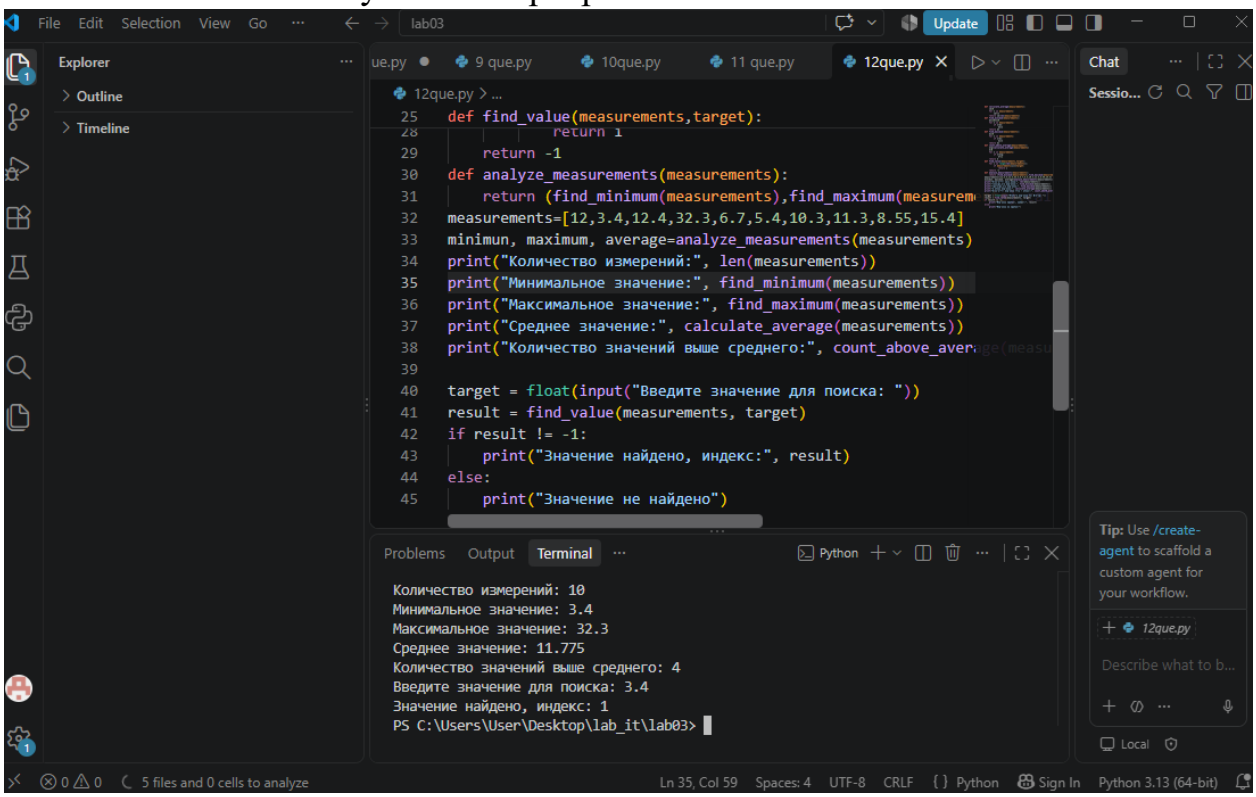


Рисунок 14: Программа 2 часть

Вывод:

В ходе выполнения лабораторной работы мы освоили ключевые конструкции языка Python и закрепили навыки структурного программирования.

Мы научились создавать собственные функции, передавать в них аргументы и обрабатывать возвращаемые значения. Мы разобрались с использованием параметров по умолчанию и отработали практику многократного вызова функций с разными данными. Мы изучили принципы области видимости переменных и убедились на практике, что локальные переменные недоступны вне своей функции, а изменение локальной переменной не затрагивает одноимённую глобальную. Мы отработали базовые операции со списками: создание, доступ к элементам по индексу, изменение значений, добавление и удаление элементов. Мы научились эффективно использовать срезы для извлечения подмножеств данных — в том числе для получения первых и последних элементов, выборки через шаг и обращения списка в обратном порядке